PROGRAM :	***SMART COMPUTING***
SUBMITTED TO :	***PROFESSOR SARWAR***
NAMES	***ID***
MUHAMMAD HAMZA SHAKEEL	***2417295***
ROSHAN AHMAD	***2417296***
HUZAIFA AHMAD QURESHI	***2417297***
WALEED SATTAR	***2417301***
NIHARIKA THAPA MAGAR	***2417135***

→DEPARTMENT OF SMART COMPUTING←

→REPORT_PROJECT ←

AI BASED DOCUMENT SIMPLIFICATION AND ACTION RECOMMENDED

ABBREVIATIONS

AI	**Artificial Intelligence**
OCR	**Optical Character Recognition**
NLP	**Natural Language Processing**
API	**Application Programming Interface**
UI	**User Interface**
PDF	**Portable Document Format**
ML	**Machine Learning**
DB	**Database**
HCI	**Human-Computer Interaction**
SRS	**Software Requirements Specification**
SDLC	**Software Development Life Cycle**

TABLE OF CONTENTS

1 INTRODUCTION 1

1.1 Brief 1
1.2 Relevance to Course Modules 1
1.3 Project Background 2
1.4 Literature Review 2
1.5 Analysis from Literature Review (in the context of your project) 3
1.6 Methodology and Software Lifecycle for This Project 3
1.6.1 The rationale behind Selected Methodology 4

2 PROBLEM DEFINITION 5

2.1 Problem Statement 5
2.2 Deliverables and Development Requirements 6
2.3 Current System (if applicable to your project) 6

3 REQUIREMENT ANALYSIS 7

3.1 Use Cases Diagram(s) 7
3.2 Detailed Use Case 8
3.3 Functional Requirements 9
3.4 Non-Functional Requirements 10

4 DESIGN AND ARCHITECTURE 11

4.1 System Architecture 11
4.2 Data Representation [Diagram + Description] 12
4.3 Process Flow/Representation 13
4.4 Design Models [along with descriptions] 14

5 IMPLEMENTATION 15

5.1 Algorithm 15
5.2 External APIs 16
5.3 User Interface 17

6 TESTING AND EVALUATION 18

6.1 Manual Testing 18
6.1.1 System testing 19
6.1.2 Unit Testing 20
6.1.3 Functional Testing 21
6.1.4 Integration Testing 22
6.2 Automated Testing 23
Tools used: 24

7 CONCLUSION AND FUTURE WORK 25

7.1 Conclusion 25
7.2 Future Work 26

8 REFERENCES 27

LIST OF FIGURES

Figure 1.1 System Overview Diagram 4

Figure 2.1 Use Case Diagram 7

Figure 3.1 System Architecture Diagram 11

Figure 3.2 Data Flow Diagram (DFD) 12

Figure 3.3 Process Flowchart 13

Figure 4.1 User Interface – Upload Page 17

Figure 4.2 User Interface – Output Dashboard 18

LIST OF TABLES

Table 2.1 Detailed Use Case 8

Table 3.1 Functional Requirements 9

Table 3.2 Non-Functional Requirements 10

Table 4.1 Data Representation 12

Table 5.1 External APIs Used 16

Table 6.1 Unit Testing 20

Table 6.2 Functional Testing 21

Table 6.3 Integration Testing 22

1. INTRODUCTION

1.1 Brief

- AI system for document understanding & action recommendation
- Targets foreign students in Korea
- Processes official documents (visa, tax, insurance, university)
- Uses OCR + NLP
- Outputs: summary, key info, actions
- Helps avoid confusion & missed deadlines

1.2 Relevance to Course Modules

This project integrates multiple computing concepts:

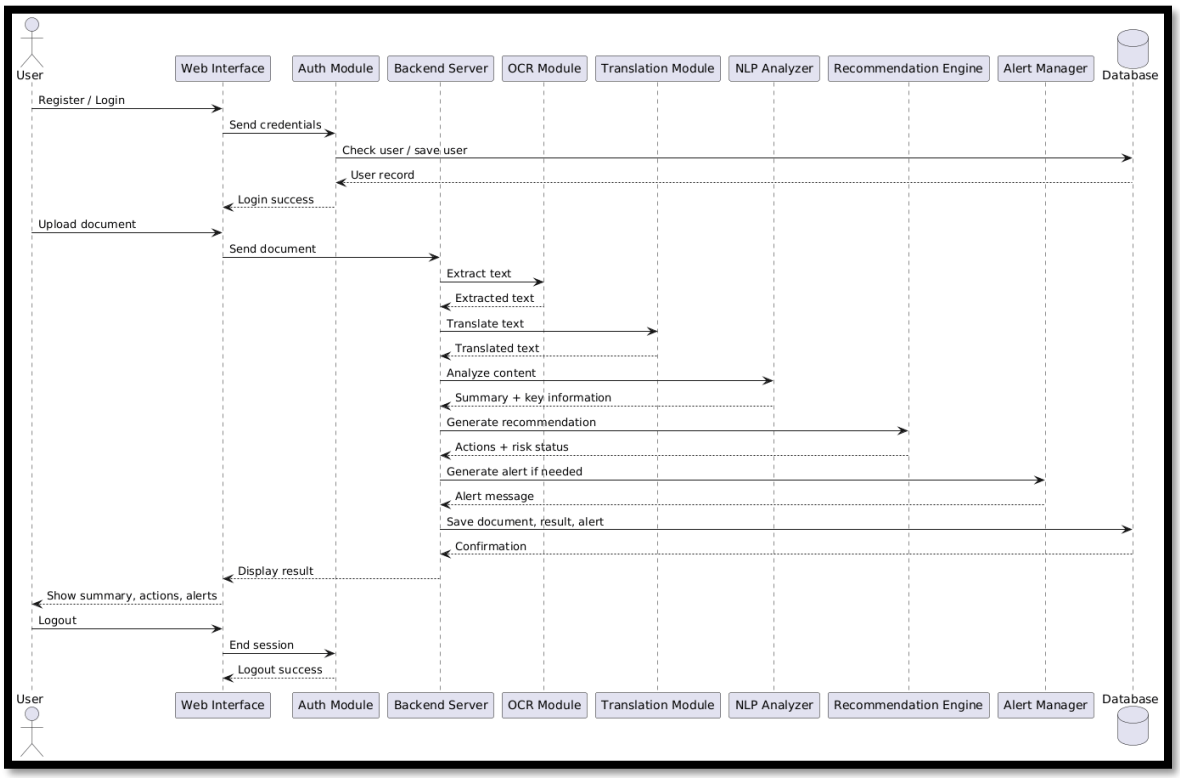
- Artificial Intelligence (NLP and classification)
- Software Engineering (system design and development lifecycle)
- Database Systems (data storage and retrieval)
- Human-Computer Interaction (user interface design)
- Web Development (frontend and backend integration)

1.3 Project Background

- Foreign students receive Korean official documents
- Hard to understand (language + complex format)
- Results in:
- Missed deadlines
- Financial penalties
- Legal issues
- Existing tools:
- Only translate/extract text
- No clear understanding
- No action guidance

1.3 Key Idea

- Simple summaries
- Clear action steps
- Deadline alerts



1.4 Literature Review

Existing systems include OCR tools, translation services, and document summarization platforms. These systems can extract text and convert it into another language. However, they do not identify important elements such as deadlines, required actions, or risks.

System	Features	Limitation
docAnalyzer	Chat with document	No action guidance
DocSimplify	Summary + extraction	No alerts
ABBYY	OCR	No intelligence
Linnk AI	Translation	No decision support
pdfFiller	Editing + summary	No structured actions

1.5 Methodology and Software Lifecycle

The project follows the Agile Software Development Model. Development is carried out in phases including planning, implementation, testing, and refinement. Each module is developed and tested iteratively.

2. PROBLEM DEFINITION

2.1 Problem Statement

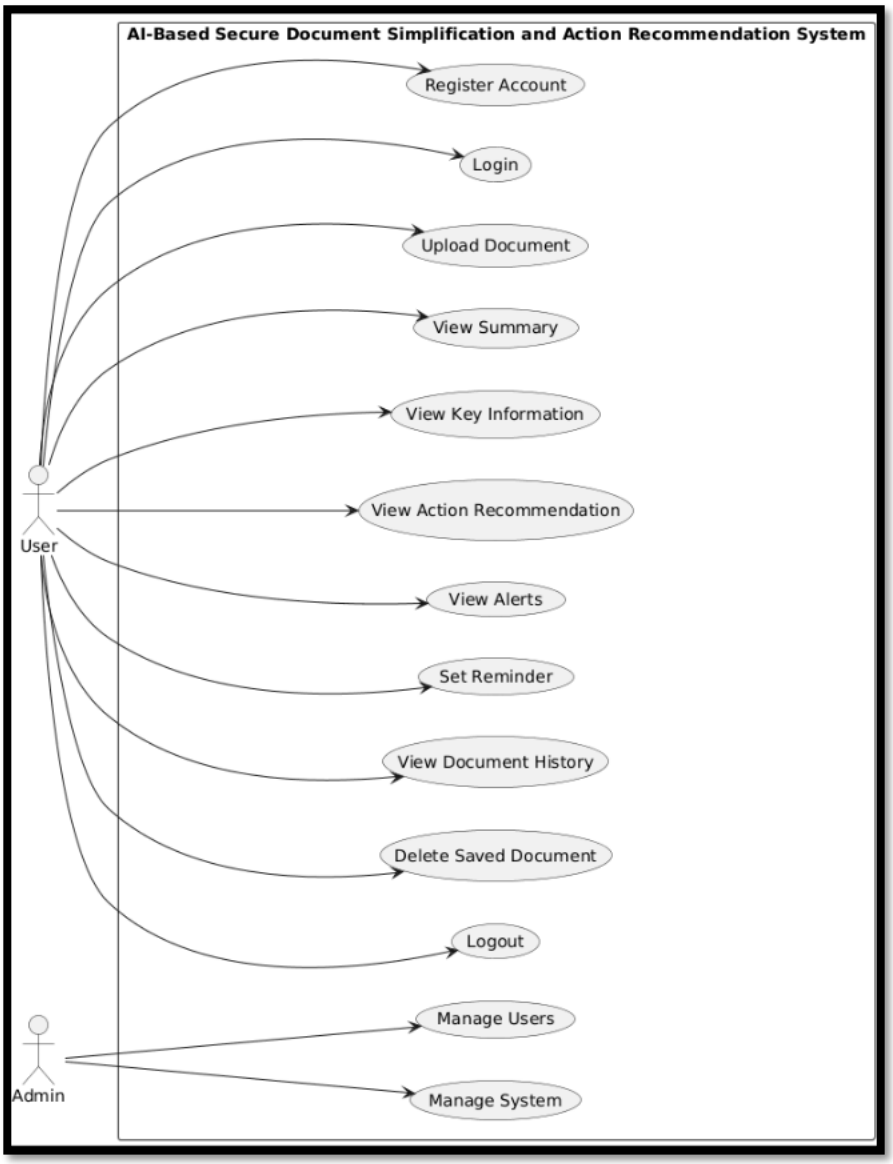
Foreign students in Korea face difficulty understanding official documents due to language barriers and complex terminology. This leads to confusion, missed deadlines, financial penalties, and stress.

2.2 Deliverables and Development Requirements

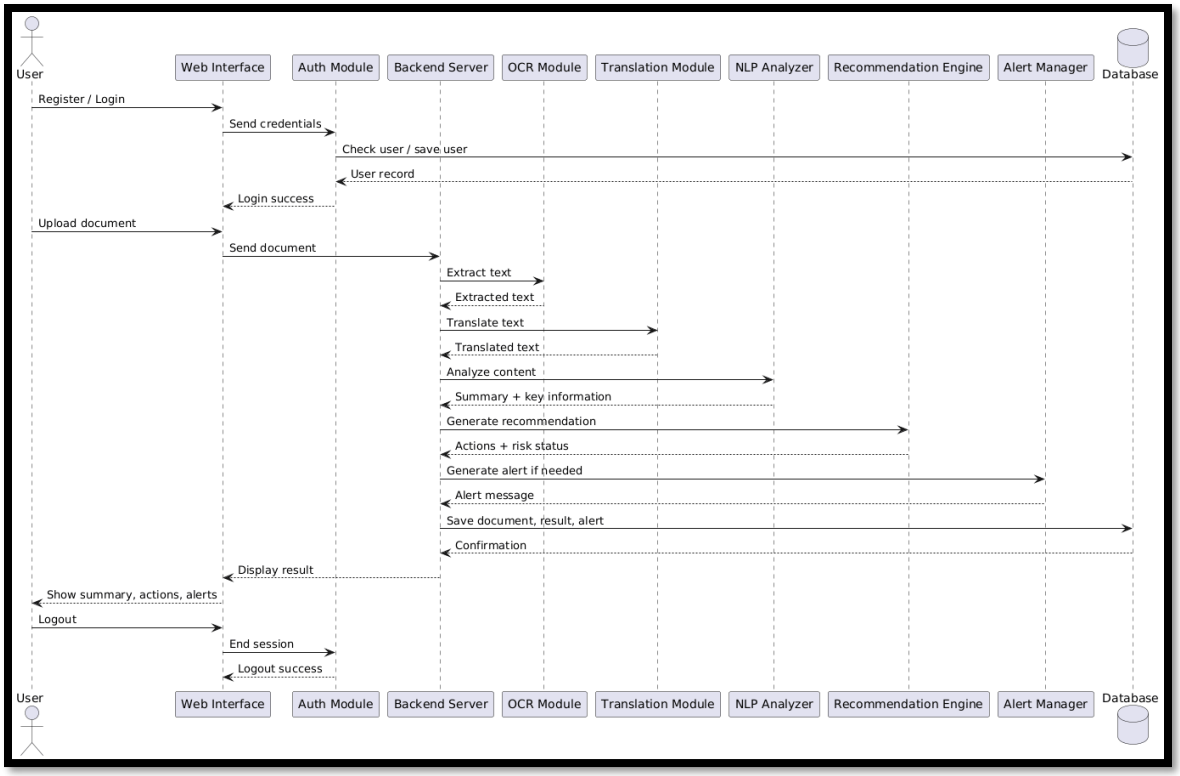
- Web-based application
- OCR module for text extraction
- NLP module for analysis
- Dashboard for displaying results
- Notification and reminder system

3. REQUIREMENT ANALYSIS

3.1 Use Case Diagram

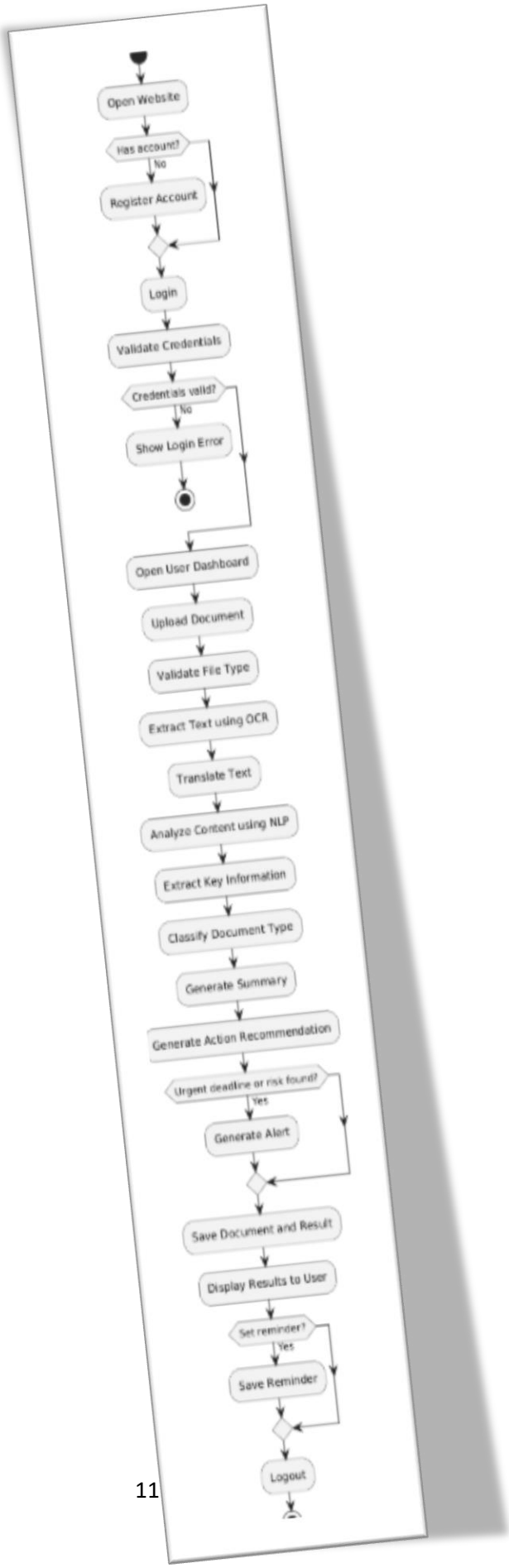


3.2 Sequential Diagram



3.3 Functional Requirements

- User registration and login
- Secure logout
- Document upload
- File validation
- OCR text extraction
- Translation
- NLP analysis
- Document classification
- Key information extraction
- Summary generation
- Action recommendation
- Alerts and reminders
- Save document history
- View and delete history
- Admin management


Open Website
Has account?
No
Register Account
Login
Validate Credentials
Credentials valid?
No
Show Login Error
Open User Dashboard
Upload Document
Validate File Type
Extract Text using OCR
Translate Text
Analyze Content using NLP
Extract Key Information
Classify Document Type
Generate Summary
Generate Action Recommendation
Urgent deadline or risk found?
Yes
Generate Alert
Save Document and Result
Display Results to User
Set reminder?
Yes
Save Reminder
Logout

3.4 Non-Functional Requirements

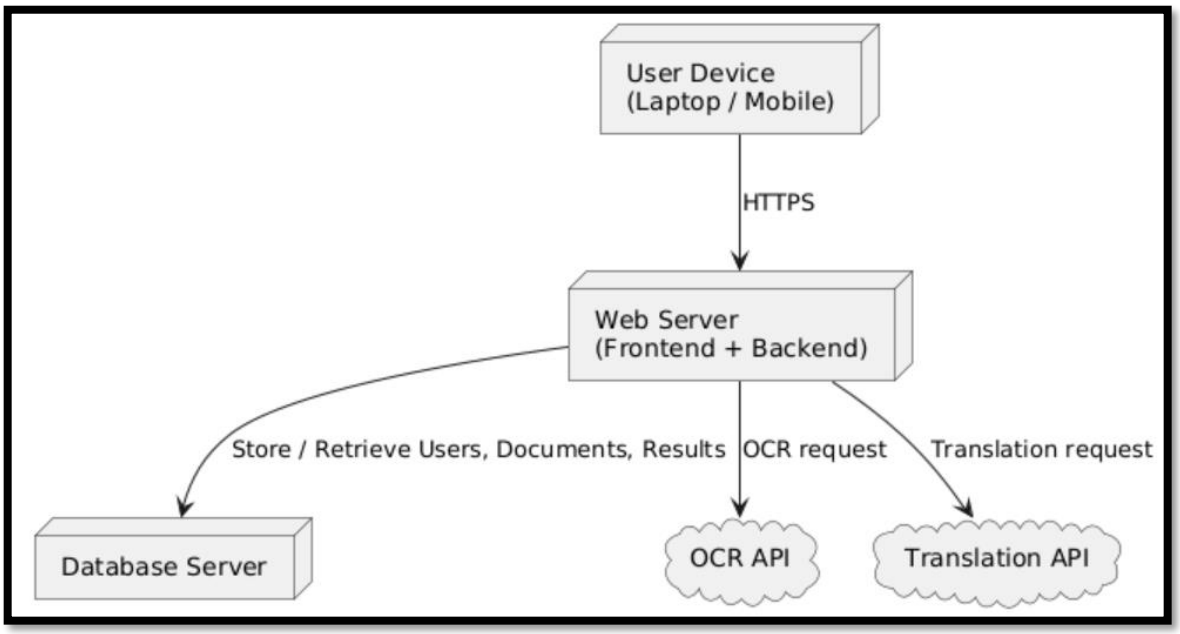
- User-friendly interface
- Fast response time
- High accuracy
- Password hashing
- Access control
- Secure storage
- File validation
- Web accessibility
- Scalability
- Maintainability
- Reliability

4. DESIGN AND ARCHITECTURE

4.1 System Architecture

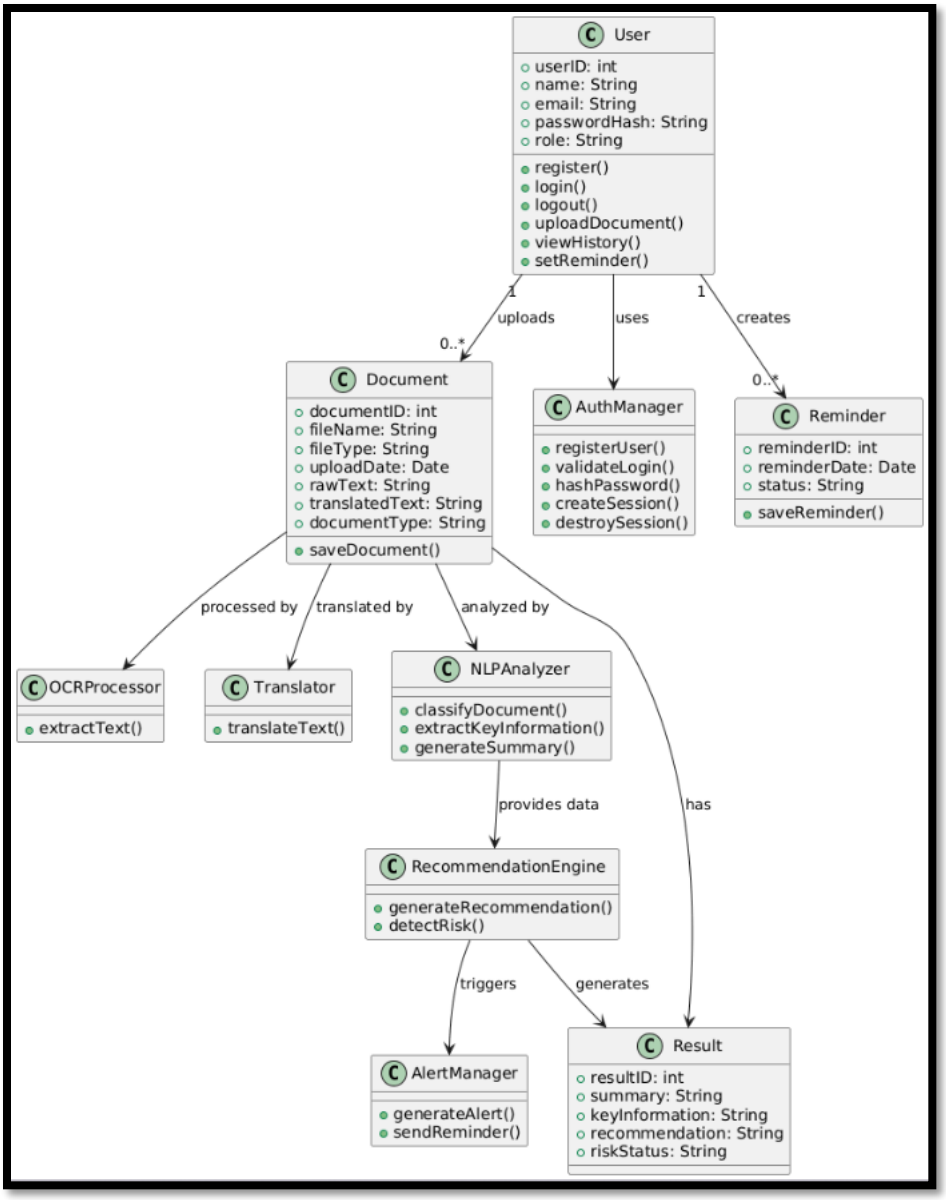
The system includes:

- Frontend (user interface)
- Backend (processing logic)
- OCR module
- NLP module
- Database



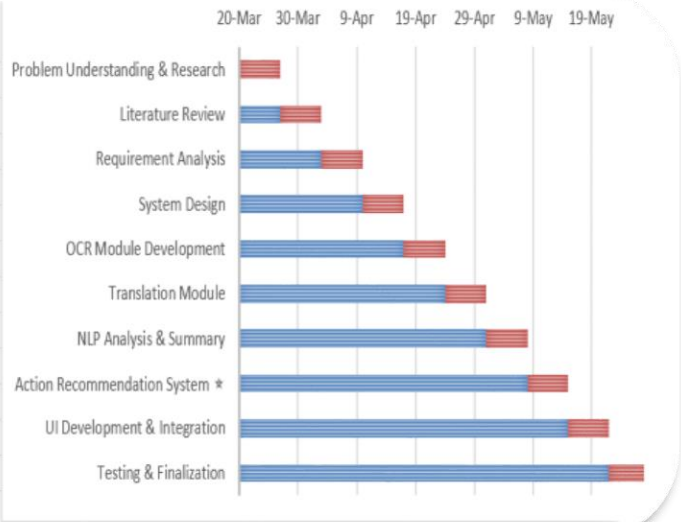
4.2 Process Flow

- User registers or logs in
- Upload document (PDF/Image)
- Validate file type and format
- Extract text using OCR
- Translate text (if required)
- Analyze content using NLP
- Extract key information (deadlines, amounts, dates)
- Classify document type
- Generate summary
- Generate action recommendations
- Check for risks or urgent deadlines
- Generate alerts (if needed)
- Save document and results to user account
- Display output (summary, actions, alerts)
- User can view history or set reminders
- User logs out



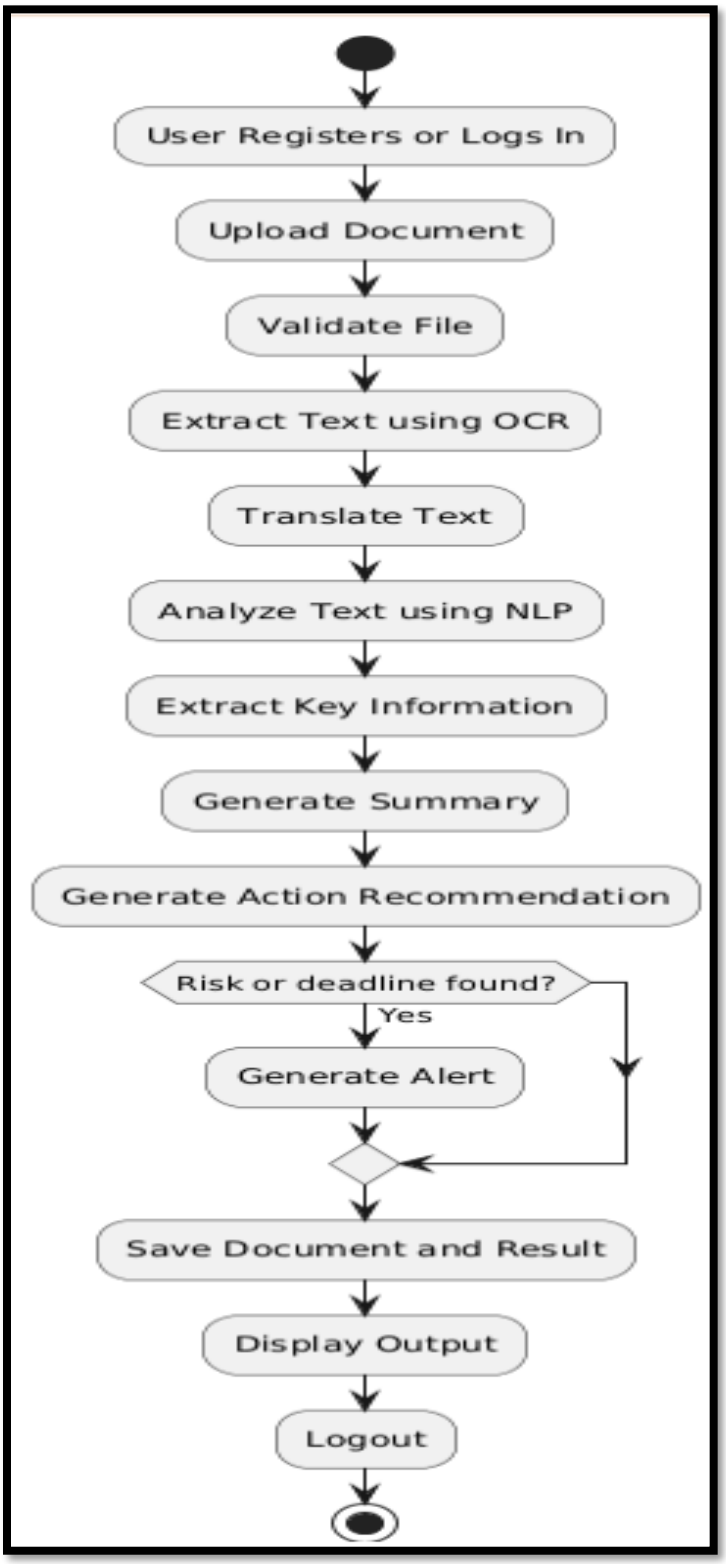
4.4 Project Timeline (Gantt Chart)

Week	Task	Start Date	End Date
1	Problem Understanding & Research	20-Mar	26-Mar
2	Literature Review	27-Mar	2-Apr
3	Requirement Analysis	3-Apr	9-Apr
4	System Design	10-Apr	16-Apr
5	OCR Module Development	17-Apr	23-Apr
6	Translation Module	24-Apr	30-Apr
7	NLP Analysis & Summary	1-May	7-May
8	Action Recommendation System *	8-May	14-May
9	UI Development & Integration	15-May	21-May
10	Testing & Finalization	22-May	28-May



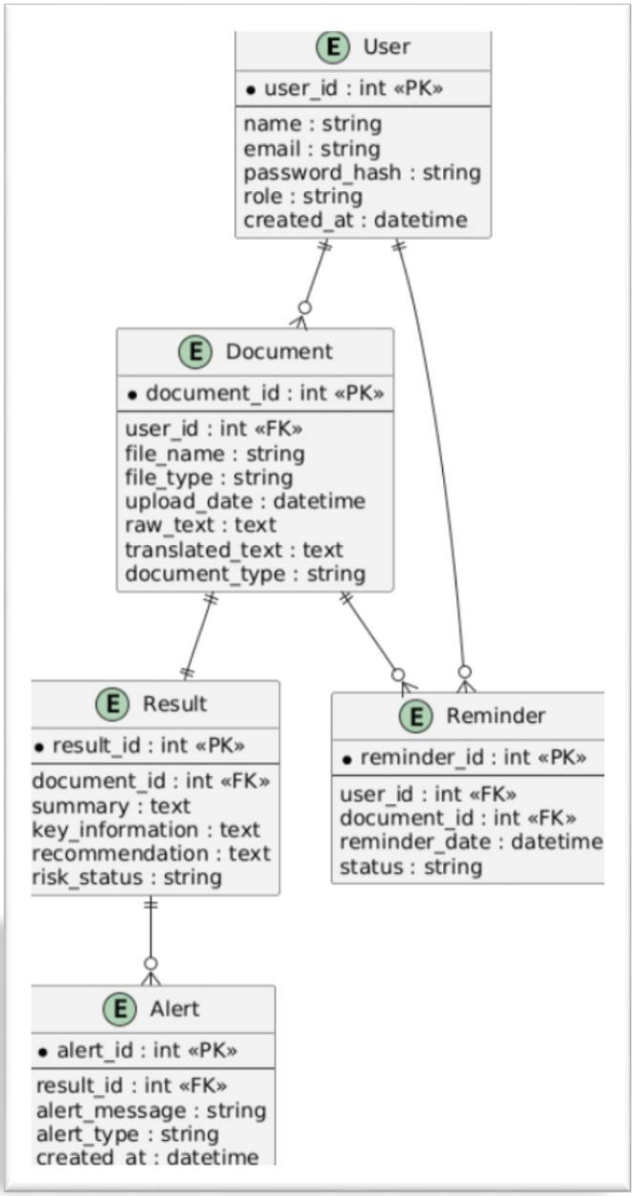
5. IMPLEMENTATION

5.1 Algorithm



5.2 External APIs

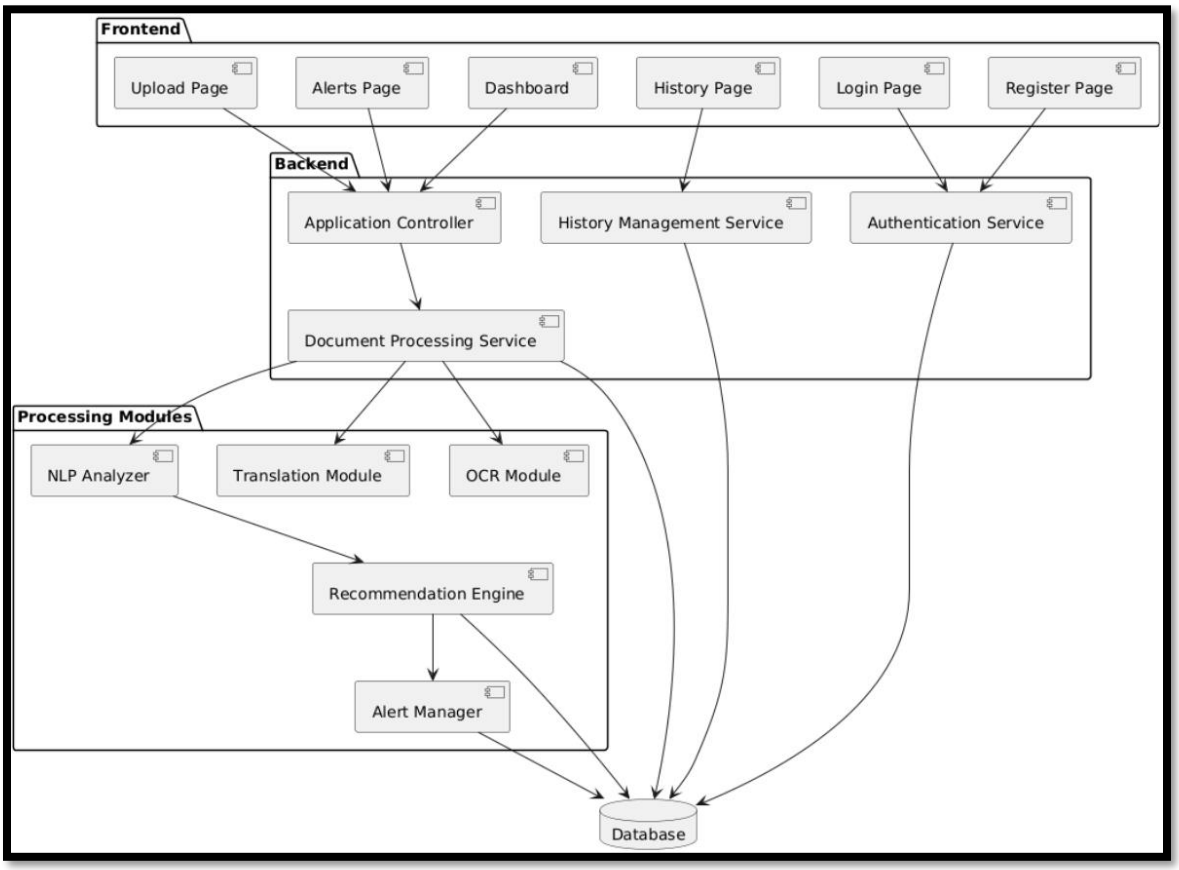
- OCR API (text extraction)
- Translation API



5.3 User Interface

The system includes:

- Register and login pages
- User dashboard
- Upload page
- Results dashboard
- Alerts and notifications
- Document history
- Reminder section
- Logout option



6. TESTING AND EVALUATION

6.1 Manual Testing

Manual testing is performed to verify system functionality through direct user interaction. It ensures that all modules such as authentication, document processing, and result generation work correctly and produce expected outputs.

6.1.1 System Testing

System testing ensures that the complete system functions correctly and meets all defined requirements.

Tested Features:

- User registration and login
- Document upload and validation
- OCR text extraction
- Translation
- NLP analysis
- Summary generation
- Action recommendation
- Alert generation
- Document saving and history

6.1.2 Unit Testing

Unit Testing 0: User Authentication

Objective: Ensure login and registration work correctly

No.	Test Case	Input	Expected Result	Result
1	Register user	Email + Password	Account created successfully	Pass
2	Login user	Valid credentials	Login successful	Pass

Unit Testing 1: Upload Document

Testing Objective: To ensure that the document upload function is working correctly.

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Verify document upload after clicking the “Upload” button with a valid PDF file	File Type: PDF, File Name: visa_notice.pdf	The document is uploaded successfully and becomes available for processing	Pass
2.	Verify document upload after clicking the “Upload” button with a valid image file	File Type: JPG, File Name: insurance_bill.jpg	The image document is uploaded successfully and becomes available for processing	Pass

Unit Testing 2: OCR Text Extraction

Testing Objective: To ensure that the OCR module extracts text correctly from uploaded documents.

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Extract text from a visa notice document	Input File: visa_notice.pdf	Text is correctly extracted from the document	Pass
2.	Extract text from a health insurance document image	Input File: insurance_bill.jpg	Text is correctly extracted from the image	Pass

Unit Testing 3: Translation Module

Testing Objective: To ensure that Korean text is translated correctly into English.

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Translate extracted Korean visa text into English	Input Text: Korean visa notice	English translation is generated correctly	Pass
2.	Translate extracted Korean insurance text into English	Input Text: Korean insurance bill	English translation is generated correctly	Pass

Unit Testing 4: Summary Generation

Testing Objective: To ensure that the summary generation module is working properly.

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Generate summary for uploaded visa document	Document Type: Visa Notice	A clear and short summary is displayed	Pass
2.	Generate summary for uploaded tax document	Document Type: Tax Notice	A clear and short summary is displayed	Pass

6.1.3 Functional Testing

The functional testing will take place after the unit testing. In this functional testing, the functionality of each module is tested. This is to ensure that the system produced meets the specifications and requirements.

Functional Testing 1: Upload and Analyze Document

Objective: To ensure that the system correctly processes an uploaded document and displays the result.

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Upload a visa document and run analysis	File Name: visa_notice.pdf	The system extracts text, translates it, and displays summary with action recommendations	Pass
2.	Upload an insurance bill and run analysis	File Name: insurance_bill.jpg	The system extracts text, translates it, and displays summary with payment deadline alert	Pass

Functional Testing 2: Display Action Recommendations

Objective: To ensure that the system shows appropriate actions based on the analyzed document.

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Analyze visa renewal notice	Document Type: Visa	Action recommendation such as “Renew visa before deadline” is displayed	Pass
2.	Analyze insurance bill	Document Type: Insurance	Action recommendation such as “Pay insurance bill before due date” is displayed	Pass

Functional Testing 3: Alert Generation

Objective: To ensure that deadline and risk alerts are generated correctly.

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Analyze tax notice with near deadline	Deadline: 3 days remaining	Alert message is displayed to warn the user	Pass
2.	Analyze overdue insurance bill	Status: Overdue	Risk warning is displayed to the user	Pass

6.1.4 Integration Testing

Ensures all modules work together correctly.

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Upload document and run OCR processing	File Name: visa_notice.pdf	Document is uploaded successfully and text is extracted correctly	Pass
2.	Translate extracted text and perform analysis	Input Text: OCR output	Translated text is analyzed and summary is generated	Pass
3.	Generate summary and action recommendations	Document Type: Insurance Bill	Correct summary, deadline, and action recommendations are shown	Pass

No.	Test case/Test script	Attribute and value	Expected result	Result
4.	Display alerts after full document analysis	Deadline: Near due date	System shows proper alert and warning message	Pass

6.2 Automated Testing

Tools used:

Tool Name	Tool Description	Applied on [list of related test cases/FR/NFR]	Results
PyTest	Python testing framework used for backend module testing	OCR module, Translation module, Summary generation	Pass
Selenium	Automation testing tool used for UI and workflow testing	Upload document, Analyze document, Display results	Pass

7. CONCLUSION AND FUTURE WORK

7.1 Conclusion

- Reduces confusion
- Prevents missed deadlines
- Supports decision-making
- Helps foreign students in Korea

7.2 Future Work

- Mobile application
- Improved AI models
- Support for more document types
- Real-time notifications

8. REFERENCES

(Add later…..)